impl InverseCDF for CanonicalRV

Aishwarya Ramasethu (@aramasethu), Yu-Ju Ku (@yuju1998), Jordan Awan, Michael Shoemate (@Shoeboxam)

May 1, 2026

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of the implementation of InverseCDF for CanonicalRV.
The implementation computes the inverse CDF of the canonical noise random variable.

1 Hoare Triple

Precondition

Compiler-verified

- Argument `self` of type CanonicalRV.
- Argument `uniform` of type `RBig`.
- Argument `refinements` of type `usize`.
- Generic `R` implements ODPRound which denotes the rounding mode, either up or down.

Caller-verified

Argument `uniform` is in $[0, 1]$.

Pseudocode

```
class InverseCDF(CanonicalRV):
    # type Edge = RBig

    def inverse_cdf(self, uniform: RBig, _refinements: usize, _R) -> RBig | None:
        f_inv = quantile_cnd(uniform, self.tradeoff, self.fixed_point)  #
        if f_inv is None:
            return None
        return f_inv * self.scale + self.shift
```

Postcondition

Theorem 1.1. Given a random variable `self` (of type CanonicalRV), the algorithm returns `Ok(Some(out))` where `out` is the inverse cumulative distribution function of `CanonicalRV` (which includes a rescale and shift) evaluated at `uniform` with error in direction `R`, returns `Ok(None)` if the quantile function returns `None`, or returns an error if evaluating the tradeoff function fails.

The error between `out` and the exactly-computed CDF decreases monotonically as `refinements` increases.

Proof. By the definition of CanonicalRV,

- `self.tradeoff` is a symmetric nontrivial tradeoff function
- `self.fixed_point` is the fixed point of `self.tradeoff`, where `tradeoff(fixed_point) = fixed_point`.

Therefore the preconditions of `quantile_cnd` are met, so `f_inv` on line 5 is a sample from the canonical noise distribution with shift of zero and scale of one. If evaluating `self.tradeoff` fails while computing `f_inv`, then the implementation propagates that error. If `quantile_cnd` returns `None`, the implementation returns `Ok(None)`.

The function then returns the outcome, scaled by `self.scale` and shifted by `self.shift`.

Computing F_f^{-1}, rescaling and shifting are exact via fractional arithmetic, so making use of refinements is not necessary, as the error in computing the inverse CDF is already zero, satisfying the monotonicity property of the error.

□